

Bridges

You are given a connected, undirected graph G with V vertices and E edges. A *bridge* of G is an edge whose removal disconnects G .

Compute the number of bridges of G .

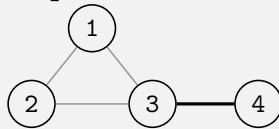
Example Test Cases:

Test Case 1

Input: $V = 4$, $E = 4$, edges = $[(1,2), (2,3), (3,1), (3,4)]$

Output: 1

Explanation:



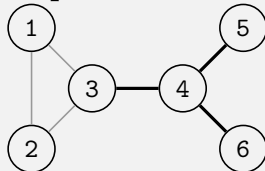
Removing the edge $(3,4)$ disconnects the graph. There are no other bridges present.

Test Case 2

Input: $V = 6$, $E = 6$, edges = $[(1,2), (2,3), (1,3), (3,4), (4,5), (4,6)]$

Output: 3

Explanation:



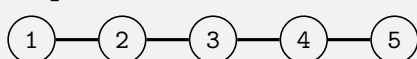
Removing the edge $(4,5)$ disconnects the graph. Similarly, edges $(3,4)$ and $(4,6)$ are also bridges. There are no other bridges present.

Test Case 3

Input: $V = 5$, $E = 4$, edges = $[(1,2), (2,3), (3,4), (4,5)]$

Output: 4

Explanation:



Removing any edge in this graph causes the resulting graph to be disconnected. Therefore, all 4 edges are bridges.

Expected Time Complexity: $O(E \cdot (V + E))$

Input Format:

An integer V , an integer E , and an array `edges` of length E . Each element of `edges` is a pair (u, v) which indicates an edge between vertex u and vertex v in G .

Output Format:

Return a single integer, the number of bridges of G .

Constraints:

1. $2 \leq V \leq 1000$
2. $1 \leq E \leq 2000$
3. $1 \leq u, v \leq V$ for all (u, v) in `edges`
4. G has no self-loops or repeated edges.

Instructions:

You have been provided with two template files, containing the `main` and `solve` functions respectively. The `main` function, which has been completed for you, parses the input for each test case. For each test case, it calls the `solve` function, which must compute the number of bridges.

The `solve` function must compute this in $O(E \cdot (V + E))$ time.

(Refer to the `Instructions.pdf` file for detailed guidelines on the input-output format and additional test cases.)

Library Usage

For this lab, you are permitted to use any modules which are part of the standard library of either C++ or python. For C++, do not add any extra `#include` directives; the provided `bits/stdc++.h` covers all allowed modules.